

Premier jet de programmation

On a commencé par s'assurer que la valeur du capteur donnée en paramètre était valide, c'est-à-dire qu'elle se trouvait entre 0 et 127 inclusivement. Ensuite, on a exploré toutes les entrées du tableau en s'assurant que le bit de validation était bien à zéro. À cette fin, on a simplement poussé un bit activé de 7 places vers la gauche afin de créer un masque sur l'entrée, qui renverrait 0 si ce bit n'était pas activé, et qui renverrait 0b10000000 sinon, déclenchant une erreur. Si ce bit de validation ne déclenchait pas d'erreur, on utilisait la fonction `Indexes` fournie dans le manuel du cours (Lemire, 2025, p. 108) pour noter la position des bits activés dans les 24 bits les plus significatifs de la ligne de données. Une fois ces positions données, on vérifiait qu'il y en avait en fait qu'une seule, c'est-à-dire qu'un seul bit dans la chaîne était activé et que la longueur du tableau donné par la fonction `Indexes` était de longueur 1, puis on augmentait la valeur dans le tableau résultant de 1 à la position de ce bit. Une fois l'intégralité du tableau entrant parcouru, on retourne le tableau de résultats contenant le nombre de fois que chaque bit était activé dans les entrées valides du capteur passé en argument. On a ensuite utilisé l'exemple donné dans l'énoncé dans la fonction `main` pour rapidement tester que la fonction fonctionnait comme prévu pour ces quelques cas. On a décidé d'implémenter que, lors du déclenchement d'une erreur, un tableau rempli de zéro soit retourné en plus de l'erreur, au lieu de ne rien retourner. Ceci pourrait être modifié selon les besoins de l'implémentation systémique plus large de la fonction.

Programmation des tests

On s'est ensuite penché sur les tests unitaires, en utilisant l'IA générative pour générer la structure générale des tests, mais pas les cas en tant que tels (Microsoft CoPilot, 2026). On a utilisé un éditeur de texte pour facilement compter les positions des caractères dans la chaîne de bit et un convertisseur binaire décimal hexadécimal (*Binary/Decimal/Hex/Octal Converter*, n.d.) pour s'assurer que les cas utilisaient des entrées et des sorties réellement correspondantes.

On a commencé par mettre en place les cas qui ne devraient pas générer d'erreur, en précisant quel bit devrait être marqué comme activé pour chaque cas. On a donc utilisé trois positions de bits activés (le plus à droite possible, le plus à

gauche possible, et quelque part au milieu des bits analysés) pour les cas où il n'y avait qu'une seule entrée et cette entrée représentait des données valides pour le capteur passé en argument. On a ensuite mis en place les tests où le tableau de données entrées contenait plusieurs entrées valides pour le capteur passé en argument, et on a réutilisé les positions des bits utilisés plus tôt pour éviter de créer des erreurs accidentelles liées au placement des bits dans l'entrée ou dans la sortie attendue. On a donc mis en place un test où trois bits différents sont activés dans les données entrantes, un test où le même bit est activé dans les trois données entrantes, et un test où un bit est activé dans deux entrées et un bit différent est activé dans la troisième entrée.

On s'est ensuite penché sur les cas qui devraient déclencher des erreurs. On a commencé par le nombre de bits activé dans les bits 8 à 31. Le cas le plus simple était celui où tout était valide, mais la donnée entrée contenait trop de bit activé dans les 24 bits les plus significatives. On a ensuite répété ce test, mais en cachant l'entrée erronée parmi des entrées complètement valides. On a répété ce test une dernière fois en invalidant deux entrées parmi plusieurs entrées valides. Pour tous ces cas, la fonction a été programmée de telle sorte à signaler quelle entrée a généré l'erreur en incluant son index dans le tableau de donnée entrante dans le message d'erreur.

On s'est ensuite penché sur les cas où le bit de validation est activé. On a utilisé le même format qu'avec le groupe précédent, soit une seule entrée erronée, une entrée erronée parmi plusieurs entrées valides, et plusieurs entrées erronées parmi plusieurs entrées valides. On s'est d'ailleurs assuré que l'erreur générée précise quelle entrée est invalide, de telle sorte que si on souhaite ensuite analyser cette entrée spécifique pour des raisons de contrôles de qualité ou autres, la tâche est facilitée puisqu'on sait où regarder.

Exploration des limites et ajout à la fonction

On a ensuite exploré les cas limites auxquels on n'avait pas pensé lors du premier jet de programmation. Tout d'abord, on a reconnu la possibilité que le tableau de données passé en argument soit vide. On a donc implémenté une vérification pour détecter ce cas et lancer une erreur au besoin au début de la fonction, puis on a ajouté un test correspondant à la batterie de tests. La réponse

du logiciel à un tableau entrant vide pourrait être modifiée selon les besoins de l'implémentation plus large de cette fonction, par exemple s'il est courant qu'aucun capteur ne génère d'observation, mais que les données collectées soient envoyées périodiquement peu importe s'il y a eu des observations. Dans ce cas, il serait problématique de générer des erreurs et d'arrêter le fonctionnement du logiciel pour une occurrence si fréquente.

On a ensuite réalisé qu'on avait oublié d'implémenter une vérification que l'identifiant du capteur des données analysées était bien celui passé en argument et qu'on ne faisait pas qu'agrèger les données de tous les capteurs dans le tableau de données entrant. On a donc implémenté cette vérification au début du for-loop utilisé pour explorer le tableau entrant, en passant à l'entrée suivante si le capteur n'est pas celui demandé lors de l'appel de la fonction. À cette fin, on a utilisé un masque (0b111111) permettant d'extraire uniquement les bits activés dans les bits 0 à 6 de l'entrée, et on a comparé cette valeur extraite à la valeur du capteur passé en argument convertie dans le même format de 32 bits. On a ensuite implémenté un test pour vérifier que si aucune des données entrantes n'avait été générée par le capteur passé en argument, le tableau retourné serait rempli de 0 et aucune erreur ne serait générée.

On en a profité pour implémenter des tests pour la valeur du capteur. Les tests visaient la génération d'une erreur si la valeur de capteur en argument est un peu plus grande que 127, beaucoup plus grande que 127. Il n'est pas nécessaire de tester pour un overflow positif (plus grand que 255) ou négatif (plus petit que 0) du format utilisé pour le capteur puisque go gère déjà la génération d'une erreur à cet effet si le nombre entré en argument est à l'extérieur des bornes valides de uint8 de [0,255].

On s'est ensuite penché sur la possibilité qu'une des entrées puisse ne contenir aucun bit activé dans les bits 31 à 8, mais que l'absence de donnée ne doive pas nécessairement générer d'erreur. On a donc implémenté une vérification pour ce cas dans la fonction principale juste après avoir vérifié que l'entrée en analyse ne contient pas plus qu'un bit activé. Cette vérification renvoie à la prochaine entrée à analyser puisqu'il n'y a pas d'information à extraire si aucun bit n'est activé. Cependant, il est possible d'implémenter facilement le déclenchement d'une erreur dans ce cas si ce comportement devait être impossible dans le système qui utilise

la fonction. On a aussi ajouté deux cas de test pour couvrir cette possibilité, soit un cas où la seule donnée dans le tableau entrant ne contient pas de bit activé dans les bits 8 à 31, et un cas où une des entrées dans un tableau en contenant plusieurs ne contient pas ce bit activé dans cet intervalle.

Finalement, on s'est penché sur la possibilité qu'aucune des données dans le tableau entrant ne corresponde au capteur passé en argument. Les vérifications déjà mises en place précédemment permettent d'assurer que ce cas ne générerait pas d'erreur et que le tableau sortant ne contiendrait que des 0. On a donc simplement ajouté des tests pour s'assurer que la sortie attendue est bien générée si la seule donnée dans le tableau entrant ne correspond pas au capteur à analyser, si toutes les données dans le tableau entrant ne correspondent pas au capteur d'intérêt, et finalement si seulement certaines données dans le tableau correspondent au capteur d'intérêt.

Bibliographie

Binary/Decimal/Hex/Octal Converter. (n.d.). Retrieved March 12, 2026, from

<https://www.rapidtables.com/convert/number/hex-dec-bin-converter.html>

Lemire, D. (2025). *Maîtriser la programmation: Des tests à la performance en Go*.

[https://m2.telug.ca/pluginfile.php/1081387/mod_resource/content/6/inf2007.](https://m2.telug.ca/pluginfile.php/1081387/mod_resource/content/6/inf2007.pdf)

pdf

Microsoft CoPilot. (2026, March 11). *Unit test generation for analyse.go file* [Chat

avec IA générative, requête : “can you generate a unit test for the

analyse.go file”]. Claude Haiku 4.5 intégré à VSCode.

Annexes

Annexe 1 – contenu du fichier analyse.go

package main

```
import (
    "fmt"
    "math/bits"
)

// Analyse traite les donnees d'un capteur specifique et retourne un tableau de
// compteurs pour les bits actifs
// dans les bits 31 a 8 des entrees valides, sachant que les entrees valides sont
// celles qui ont le 7e bit a 0
// et dont les bits 0 a 6 correspondent a l'identifiant du capteur specifique
func Analyse(data []uint32, capteur uint8) ([24]int, error) {
    var result [24]int
    //verifie que les donnees ne sont pas vides
    if len(data) == 0 {
        return [24]int{}, fmt.Errorf("Aucune donnee a analyser")
    }
    //verifie que la valeur capteur est valide
    if capteur > 127 {
        return [24]int{}, fmt.Errorf("Capteur invalide, doit etre entre 0 et 127")
    }
    for index, value := range data {
        // verifie l'identifiant du capteur dans les bits 0 a 6
        if value&0x7F != uint32(capteur) {
            continue //identifiant du capteur ne correspond pas, on passe a l'entree
            suivante
        }
        //verifie que le 7e bit est a 0
        if value&(1<<7) != 0 {
            return [24]int{}, fmt.Errorf("Bit de validation invalide, le 7e bit est a 1 dans
l'entree a l'index %d", index)
        }
        //note les positions des bits activees dans les bits 31 a 8
        index := Indexes(value >> 8)
        //verifie que juste un bit est active dans les bits 31 a 8
        if len(index) > 1 {
            return [24]int{}, fmt.Errorf("Plus d'un bit actif dans les bits 31 a 8 dans
l'entree a l'index %d", index)
        } else if len(index) == 0 {
            continue //si aucun bit active, on passe a l'entree suivante, pourrait
            declancher une erreur selon les besoins
        }
        //ajoute un a la position correspondante dans le resultat
        result[index[0]]++
    }
}
```

```
    return result, nil
}

// tel que disponible dans le manuel du cours p108, modifier de 64 bits a 32 bits
// pour correspondre au type de données utilisé dans ce projet
func Indexes(x uint32) []int {
    var ind = make([]int, bits.OnesCount32(x))
    pos := 0
    for x != 0 {
        ind[pos] = bits.TrailingZeros32(x)
        x &= x - 1
        pos++
    }
    return ind
}

func main() {
    data := []uint32{
        0x00000105, // ID=5, bit 7=0, bit 8=1 (valide)
        0x00000205, // ID=5, bit 7=0, bit 9=1 (valide)
        0x00000080, // ID=0, bit 7=1 (invalide)
        0x00000305, // ID=5, bit 7=0, bits 8 et 9=1 (invalide)
    }
    counts, err := Analyse(data, 5)
    if err != nil {
        // Résultat attendu : err != nil (à cause des entrées invalides)
        fmt.Println("Erreur lors de l'analyse:", err)
    } else {
        // Si aucune erreur : counts = [1, 1, 0, 0, ..., 0] (bit 8: 1 fois, bit 9: 1 fois)
        fmt.Println("Analyse réussie, counts:", counts)
    }
}
```

Annexe 2 – contenu du fichier analyse_test.go

package main

```
import (
    "testing"
)
```

```
func TestAnalyse(t *testing.T) {
    tests := []struct {
        name    string
        data    []uint32
        capteur uint8
        expected [24]int
        wantErr bool
    }{
```

```

    {
        name: "une entree valide, bit 8 active",
        data: []uint32{0x00000105}, // bit 8 active
        capteur: 5,
        expected: [24]int{1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 0},
        wantErr: false,
    },
    {
        name: "une entree valide, bit 31 active",
        data: []uint32{0x80000005}, // bit 31 active
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 1},
        wantErr: false,
    },
    {
        name: "une entree valide, bit 16 active",
        data: []uint32{0x00010005}, // bit 16 active
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 0},
        wantErr: false,
    },
    {
        name: "plusieurs entrees valides, bits differents actives",
        data: []uint32{0x00000105, 0x00010005, 0x80000005}, // bits 8, 16, 31
actives
        capteur: 5,
        expected: [24]int{1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 1},
        wantErr: false,
    },
    {
        name: "plusieurs entrees valides, memes bits actives",
        data: []uint32{0x00010005, 0x00010005, 0x00010005}, // bit 16 active 3
fois
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 0},
        wantErr: false,
    },
    {
        name: "plusieurs entrees valides, memes bits et bits differents actives",
        data: []uint32{0x00010005, 0x00010005, 0x80000005}, // bit 16 active 2
fois, bit 31 active 1 fois
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
0, 1},

```

[illegible]

```

    {
        name: "trop de bits actives dans une entree",
        data: []uint32{0x80010005}, // bit 16 et 31 active
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        wantErr: true,
    },
    {
        name: "trops de bits actives dans une entree parmi plusieurs entrees valides",
        data: []uint32{0x00010005, 0x00010005, 0x80010005}, // bit 16 active 2 fois, bit 31 active 1 fois
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        wantErr: true,
    },
    {
        name: "trops de bits actives dans plusieurs entrees parmi plusieurs entrees valides",
        data: []uint32{0x00010005, 0x00010005, 0x00010105, 0x80010005}, // bit 16 active 2 fois, puis 8 et 16 dans une entree, et 31 et 16 dans une autre entree
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        wantErr: true,
    },
    {
        name: "un bit de validation invalide pour une entree",
        data: []uint32{0x80000085}, // bit 7 active
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        wantErr: true,
    },
    {
        name: "un bit de validation invalide pour plusieurs entree",
        data: []uint32{0x00010005, 0x00010085, 0x80000005}, // bit 7 active dans deuxieme entree
        capteur: 5,
        expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        wantErr: true,
    },
    {
        name: "plusieurs bits de validation invalides pour plusieurs entree",
        data: []uint32{0x00010085, 0x00010085, 0x80000005, 0x00000105}, // bit 7 active dans premiere et deuxieme entree

```

```

    capteur: 5,
    expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    wantErr: true,
},
{
    name: "aucune donnees",
    data: []uint32{},
    capteur: 5,
    expected: [24]int{},
    wantErr: true,
},
{
    name: "valeur de capteur invalide (un peu plus grand que 127)",
    data: []uint32{0x00000105, 0x00010005, 0x80000005}, // bits 8, 16, 31
    capteur: 128, // capteur invalide
    expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    wantErr: true,
},
{
    name: "valeur de capteur invalide (beaucoup plus grand que 127)",
    data: []uint32{0x00000105, 0x00010005, 0x80000005}, // bits 8, 16, 31
    capteur: 250, // capteur invalide
    expected: [24]int{0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    wantErr: true,
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        result, err := Analyse(tt.data, tt.capteur)
        if (err != nil) != tt.wantErr {
            t.Errorf("Analyse() error = %v, wantErr %v", err, tt.wantErr)
            return
        }
        if result != tt.expected {
            t.Errorf("Analyse() = %v, want %v", result, tt.expected)
        }
    })
}

```